

ARP & DNS Poisoning Man-in-the-Middle Attacks

From Layer-2 spoofing to DNS cache poisoning — theory, live lab walkthrough, tooling, and blue-team countermeasures

MITM

ARP SPOOFING

DNS SPOOFING

ETTERCAP

DEFENSE

Prepared for	Goraya — Junior Pentester Track
Trainer reference	Umar Niaz / CyberStr curriculum
Source	ITProTV lecture — ARP & DNS Poisoning
Level	Foundational → Junior Pentester
Lab stack	Kali Linux, Windows victim, Ettercap/Bettercap

01 Why These Attacks Even Work

CORE IDEA

ARP and DNS were designed in an era where the network was assumed to be a room full of trusted machines. Neither protocol has built-in authentication — nothing checks "is this response actually coming from who it claims to be from." Convenience was prioritized over security, and that trade-off is exactly what a pentester (or attacker) leverages.

This is the single most important mental model for this whole topic: **the more convenient/automatic a protocol is, the more it tends to trust unauthenticated input.** ARP auto-updates on any reply it receives. DNS resolvers cache whatever they're told. Both behaviors exist purely for speed and convenience — and both are exploitable.

Where This Fits in a Pentest

ARP and DNS poisoning are almost always used to set up a **Man-in-the-Middle (MITM) position** — you're not the endpoint, but you make yourself sit in the traffic path so you can:

- **Sniff** credentials, session tokens, and plaintext protocols (Telnet, HTTP, FTP)
- **Modify** traffic in transit (inject payloads, downgrade HTTPS, alter responses)
- **Redirect** victims to attacker-controlled infrastructure (fake login portals)
- **Deny service** by black-holing traffic instead of forwarding it

JUNIOR-LEVEL TAKEAWAY

Whenever an engagement scope includes "internal network" or "same broadcast segment," ARP poisoning should be on your checklist as a way to escalate from passive recon to active interception. It's usually one of the first active attacks you run once you've confirmed you're on a flat, unsegmented VLAN.

02 ARP Poisoning — Theory

2.1 What ARP Actually Does

Address Resolution Protocol maps a Layer 3 address (IPv4) to a Layer 2 address (MAC). It only matters **inside a single broadcast domain / subnet** — once traffic needs to leave the LAN, routing (not ARP) takes over. If two hosts are plugged into the same switch, they don't need to know each other's IP at all to talk — MAC addressing at Layer 2 is enough. ARP exists purely to let IP-based applications find the right MAC to hand frames to.

The ARP Resolution Flow

1. Host A wants to talk to 192.168.1.50 but only knows the IP.
2. Host A **broadcasts** an ARP request: "Who has 192.168.1.50? Tell 192.168.1.10."
3. Every host on the segment receives the broadcast; only the owner of .50 replies.
4. The reply is unicast back: "192.168.1.50 is at AA:BB:CC:DD:EE:FF."
5. Host A caches this mapping in its **ARP table**.

THE VULNERABILITY

There is no verification step. Any machine on the segment can send an unsolicited ARP reply ("gratuitous ARP") claiming to own an IP address it doesn't, and most operating systems will accept it and overwrite the existing entry. **The last reply received wins** — this is the entire basis of ARP poisoning / ARP spoofing.

Static vs Dynamic ARP Entries

Type	How it's created	Poisoning risk
Static	Manually configured by an admin	Not vulnerable — but rarely used outside critical infra (gateways, servers)
Dynamic	Learned automatically from ARP replies on the wire	Fully vulnerable — this is the default on almost every host

Check a host's ARP table with `arp -a` (Windows/Linux) — dynamic entries are the ones you can poison.

2.2 What "Poisoning" Means in Practice

To become the man in the middle between Victim A and Victim B (usually a client and the default gateway), the attacker sends forged ARP replies to *both* parties:

- To Victim A: "The gateway's IP is now at **my** MAC address."
- To Victim B (the gateway): "Victim A's IP is now at **my** MAC address."

Both machines now send their traffic to the attacker first. As long as the attacker enables **IP forwarding**, they silently relay the traffic onward after inspecting/modifying it — the classic **bidirectional (two-way) ARP poisoning** MITM setup.

PRACTICAL NOTE FROM THE LAB

One-way poisoning (only telling the victim the gateway's fake MAC) lets you see outbound traffic but breaks return traffic unless you also poison the gateway. Always poison **both directions** in an ettercap/bettercap target selection (Target 1 = victim, Target 2 = gateway) — this matches doing "OK" on ettercap's bidirectional ARP poisoning prompt instead of choosing one-way.

03 ARP Poisoning — Hands-On Lab Walkthrough

LAB TOPOLOGY (FROM THE RECORDING)

Attacker: Kali Linux. Victims: a Windows machine and another Linux host on the same subnet, communicating over Telnet (deliberately insecure/plain text for the demo). Tools: Wireshark for capture, Ettercap (GUI) for the ARP poisoning + sniffing.

3.1 Confirm ARP Traffic First (Recon Step)

- 1 Launch Wireshark and select the correct interface: `sudo wireshark`
- 2 Filter for `arp` to confirm you can see "Who has X? Tell Y" broadcasts on the wire — this proves you're in the same broadcast domain as your targets.
- 3 On a target host, run `arp -a` and note the current gateway MAC — you'll compare this *after* the attack to prove poisoning worked.

3.2 Baseline: Prove Traffic Is NOT Visible Yet

Before poisoning, generate some cross-host traffic (e.g. a Telnet login from one victim to another) and confirm in Wireshark that you **cannot** see the Telnet session — only broadcast ARP noise. This baseline is important in a real report: it demonstrates the attack had an actual effect rather than traffic being visible anyway (e.g. via a hub or mirrored port).

3.3 Run the ARP Poisoning Attack (Ettercap GUI)

- 1 Launch Ettercap graphically: `sudo ettercap -G`
- 2 Select your listening interface, then start **Sniff** → **Unified Sniffing**.
- 3 Run **Hosts** → **Scan for Hosts**, then open **Hosts** → **Host List** to see all discovered IPs.
- 4 Select the first victim IP → **Add to Target 1**. Select the second victim (often the gateway) → **Add to Target 2**.
- 5 Go to **Mitm** → **ARP Poisoning**. When prompted, tick "**Sniff remote connections**" and leave one-way *unchecked* to poison bidirectionally, then click OK.
- 6 Verify: on the victim, re-run `arp -a`. The gateway/peer IP should now resolve to the attacker's MAC address instead of the real one.
- 7 Reproduce the earlier Telnet login. In Wireshark, right-click the TCP stream → **Follow** → **TCP Stream** to view the captured credentials in cleartext.

- 8 When done, stop the MITM attack in Ettercap — this sends corrective ARP replies to restore the victims' real tables (don't skip this in an authorized engagement; leaving hosts poisoned after your test window is bad practice).

COMMON LAB GOTCHA

If poisoning "succeeds" (ARP table changes) but you still see no application traffic, check that the victim's traffic is actually routed the way you expect — e.g. the machine may have a hardcoded alternate DNS/gateway, or you poisoned the wrong pair of targets. Re-verify with `ipconfig /all` / `ip route` on the victim before assuming the tool is broken.

3.4 Command-Line Alternative (no GUI)

Ettercap and Bettercap both have CLI/text modes useful for headless boxes or automation. A rough equivalent workflow using classic `arpspoof` (dsniff suite):

```
# Enable IP forwarding first so traffic still reaches its destination
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward

# Poison the victim: tell it you are the gateway
sudo arpspoof -i eth0 -t <victim_ip> <gateway_ip>

# In a second terminal, poison the gateway: tell it you are the victim
sudo arpspoof -i eth0 -t <gateway_ip> <victim_ip>

# Capture in parallel
sudo tcpdump -i eth0 -w capture.pcap host <victim_ip>
```

04 DNS Poisoning — Theory

4.1 What DNS Does (One Layer Up From ARP)

Where ARP maps IP → MAC, DNS maps **hostname** → **IP**. Conceptually it's the same trust problem one layer higher in the stack: if you can control what IP a hostname resolves to, you control where the victim's traffic actually goes — even if their browser bar still says "facebook.com."

4.2 The DNS Resolution Order — Where You Can Interfere

This order matters because **every step is a separate attack surface**, and several of them never even leave the victim's own machine:

#	Step	Attacker angle
1	Is the query for my own hostname?	Rarely useful to attack directly
2	Check local DNS resolver cache	Cache poisoning — inject a fake record before the real one is cached
3	Check local hosts file (<code>/etc/hosts</code> or <code>C:\Windows\System32\drivers\etc\hosts</code>)	If you get local/admin access, one line here silently redirects a domain forever
4	Query configured DNS server	On-path attacker intercepts/spoofs the reply before the legit server's answer arrives (this is what the ettercap <code>dns_spoof</code> plugin does)

WHY THIS ORDER MATTERS FOR A PENTESTER

Notice that DNS spoofing over the network (step 4) is only *one* option. If you already have local code execution or admin rights on a box, editing the hosts file (step 3) is quieter and doesn't require you to stay in a MITM position at all — it survives even if you leave the network.

4.3 DNS Spoofing Requires ARP Poisoning First (Usually)

To intercept and forge DNS *responses* in transit, you need to already be positioned as the man in the middle — which means ARP poisoning is normally the prerequisite step, and DNS spoofing is layered on top of it. This is exactly the order followed in the lab: ARP poison first, confirm it worked, *then* enable DNS spoofing.

05 DNS Poisoning — Hands-On Lab Walkthrough

5.1 Configure Fake DNS Records

Ettercap's `dns_spoof` plugin reads its fake records from a config file. Edit it before starting the attack:

```
sudo nano /etc/ettercap/etter.dns

# Add A-record entries at the top of the file, format:
# <domain>      A      <fake_ip>
facebook.com    A      192.168.241.131
www.facebook.com A      192.168.241.131
twitter.com     A      192.168.241.131
```

The IP you map to should be the attacker's own machine (or wherever you're hosting a credential-harvesting clone page).

5.2 Run the Attack End-to-End

- 1 Start Ettercap GUI, unified sniffing, scan for hosts (same as the ARP walkthrough).
- 2 Add victim → Target 1, gateway → Target 2.
- 3 Open **Plugins** → **Manage the Plugins**, find `dns_spoof`, and double-click / right-click → **Activate**.
- 4 Start **Mitm** → **ARP Poisoning** exactly as before — DNS spoofing rides on top of the existing ARP MITM position.
- 5 On the victim, clear the DNS cache and confirm the DNS server in use: `ipconfig /flushdns` then `ipconfig /all`.
- 6 Trigger a resolution: `ping facebook.com` from the victim.
- 7 Verify the poison took effect: `ipconfig /displaydns` on the victim should now show facebook.com resolving to the attacker's IP.

REAL GOTCHA HIT IN THE RECORDING

The attack initially failed because the victim was hardcoded to a DNS server that was offline at the time (not the default gateway). ARP poisoning was confirmed working, but DNS spoofing wasn't triggering because queries were going to a dead server, not through the poisoned path. **Lesson: always confirm which DNS server the victim is actually configured to use before assuming your `dns_spoof` plugin will see the query.** If it's a static/off-path DNS server, you may need to also intercept traffic destined to that server's IP, not just the default gateway.

5.3 From Redirection to Impact

Once facebook.com resolves to your machine, the actual attack payoff is usually a **cloned login page** (credential harvesting) hosted on the attacker box, e.g. with a tool like **SET (Social-Engineer Toolkit)** or a simple hosted HTML clone + a listener to capture POST data. DNS spoofing gets the victim to the door; the phishing page is what actually steals the credentials.

06 Tooling Reference — Beyond What Was Demoed

The lecture used Ettercap. As a junior pentester you should be comfortable with the wider toolkit, because different engagements/EDR setups will make one tool noisier or stealthier than another.

6.1 ARP Poisoning Tools

Tool	Style	Notes
Ettercap	GUI + CLI	Used in the lecture. Plugin ecosystem (dns_spoof, etc). Good for beginners.
Bettercap	CLI, modular, REST API	The modern successor most pros are moving to — modules for ARP, DNS, WiFi, BLE, HTTP/HTTPS proxying, and a scriptable caplet system. Worth learning even though the lecture stuck with Ettercap.
arpspoof (dsniff suite)	CLI, minimal	Does one thing — send forged ARP replies. Great for scripting and understanding fundamentals.
macof	CLI	Floods a switch's CAM table to force it into fail-open/hub mode — a blunt alternative to ARP poisoning on some switches.
Cain & Abel	Windows GUI (legacy)	Historically popular for ARP poisoning + password recovery on Windows; largely superseded today but still referenced in older material/certs.

6.2 DNS Poisoning / Spoofing Tools

Tool	Style	Notes
ettercap dns_spoof plugin	Plugin	Used in the lecture; needs <code>etter.dns</code> populated with fake A records.
bettercap dns.spoof module	CLI module	Equivalent functionality, more flexible domain matching (wildcards) and easier automation.
dnsspoof (dsniff suite)	CLI	Lightweight, config-file driven like ettercap's, good for scripted labs.
Responder	CLI	Not classic DNS spoofing, but poisons LLMNR/NBT-NS/mDNS — the fallback name-resolution protocols Windows uses when DNS fails. Extremely common in real internal AD pentests for harvesting NTLM hashes.
mitm6	CLI	Abuses IPv6 (which is enabled by default on most Windows hosts but rarely defended) to become the default IPv6 DNS server, then relays to escalate against Active Directory. A modern, very effective evolution of the same "control name resolution" idea.

ADD THESE TO YOUR TOOLBOX NEXT

If you only take one extra tool away from this topic, make it **Responder** — LLMNR/NBT-NS poisoning is one of the highest-yield, lowest-effort attacks on real internal Windows networks and is a near-universal finding on internal pentests. **mitm6** is the natural next step once you're comfortable with Responder.

6.3 Supporting Tools for the Full Attack Chain

- **Wireshark / tcpdump** — capture and verify what you actually intercepted (used throughout the lecture).
- **SSLstrip / bettercap's HSTS-bypass modules** — downgrade HTTPS to HTTP once you're MITM, since plaintext sniffing alone won't get you much on modern sites.
- **SET (Social-Engineer Toolkit)** — quickly stand up a cloned login page to pair with your DNS spoof for credential harvesting.
- **Driftnet / urlsnarf** — quick visual/URL sniffing while a MITM session is live, useful for scoping what a victim is doing in real time.

07 Defensive Strategies & Mitigations

A junior pentester who can only attack is half a pentester — you need to be able to write the "Remediation" section of the report too. This is also exactly what a blue team / network engineer would deploy against everything demonstrated above.

7.1 Mitigating ARP Poisoning

LAYER 2 CONTROLS

- **Dynamic ARP Inspection (DAI)** — a Cisco/enterprise switch feature that validates ARP packets against a trusted DHCP snooping binding table and drops forged replies. This is the single most effective fix and should be top of any remediation list.
- **DHCP Snooping** — required as a prerequisite for DAI; also stops rogue DHCP servers from handing out malicious gateway/DNS info in the first place.
- **Port Security** — limit/lock which MAC addresses are allowed on a switchport, making it harder for an attacker's NIC to blend in or spoof multiple identities.
- **Static ARP entries** — realistic only for a small number of critical hosts (gateways, servers), not scalable network-wide, but effective where used.
- **Network segmentation / VLANs** — ARP poisoning only works within a broadcast domain, so smaller, well-segmented VLANs shrink the blast radius dramatically.

HOST / MONITORING CONTROLS

- **ARP monitoring tools** (e.g. `arpwatch`, XArp) — alert when an IP-to-MAC mapping changes unexpectedly.
- **IDS/IPS signatures** for excessive gratuitous ARP traffic.
- Encrypt everything possible at the application layer (TLS) so that even a successful MITM only captures ciphertext.

7.2 Mitigating DNS Poisoning

PROTOCOL-LEVEL FIXES

- **DNSSEC** — cryptographically signs DNS records so resolvers can verify a response actually came from the authoritative server and wasn't forged in transit.
- **DNS over HTTPS (DoH) / DNS over TLS (DoT)** — encrypts the query/response so an on-path attacker can't read or trivially forge it (though this doesn't help if the attacker is already the DNS server being queried, e.g. via DHCP spoofing).

- **Restrict/lock the hosts file** — file integrity monitoring or permissions that prevent unauthorized edits to `/etc/hosts` .

OPERATIONAL FIXES

- Since DNS spoofing in this lecture rode on top of ARP poisoning, **fixing ARP (DAI/DHCP snooping) removes the prerequisite** for most on-path DNS spoofing.
- Disable **LLMNR and NBT-NS** via Group Policy on Windows networks to close the Responder attack path.
- Disable IPv6 if it's unused, or properly secure/monitor it, to close the mitm6 attack path — "IPv6 is off so it's not a risk" is a common and dangerous misconception.
- Use short DNS cache TTLs judiciously and monitor for unexpected record changes on internal resolvers.

08 Extending the Concept — Load Balancers & Gratuitous ARP

WHY THIS SECTION EXISTS

The lecture didn't cover load balancers directly, but ARP poisoning and load-balancer/high-availability failover are built on the exact same underlying mechanism: **gratuitous ARP**. Understanding this connection is what separates a junior who memorized a tool from one who understands the protocol well enough to spot risk anywhere it shows up.

8.1 The Connection: How Load Balancers Use ARP

Many software load balancers and HA (high-availability) clustering setups — **keepalived (VRRP)**, **Linux Virtual Server (LVS)**, Cisco **HSRP**, and hardware balancers like F5/HAProxy in active-passive pairs — use a **Virtual IP (VIP)** shared between two or more nodes. When the active node fails and a standby takes over, the standby announces itself to the network using the *same trick an attacker uses*: it sends a gratuitous ARP broadcast claiming "the VIP is now at my MAC address," so every switch and host on the segment updates instantly without waiting for their ARP cache to expire.

THE SECURITY IMPLICATION

Because failover legitimately relies on unsolicited/gratuitous ARP, a network that hasn't deployed Dynamic ARP Inspection is **equally vulnerable to an attacker impersonating the load balancer's VIP** as it is to any other ARP spoofing attack. An attacker who poisons the VIP's MAC mapping can silently intercept all traffic meant for a "highly available" service — ironically using the same mechanism that makes it highly available in the first place.

8.2 Practical Strategy for Testing / Hardening Load-Balanced Environments

1. **Recon:** identify VIPs during enumeration (an IP that shows a changing MAC owner over time, or responds inconsistently across scans, may be a floating/HA VIP worth flagging).
2. **Test:** in an authorized engagement, attempt to ARP-poison the VIP the same way you would a gateway — if DAI/port security isn't in place, you can potentially hijack traffic meant for a "resilient" service.
3. **Recommend:** the same core fixes as Section 7.1 — DAI + DHCP snooping — but specifically scoped to include the VLAN(s) where HA/load-balancer failover traffic lives, since these are sometimes overlooked/whitelisted "because it's infrastructure, not a normal host."
4. **Segmentation:** keep load-balancer/failover VLANs isolated from general user VLANs so that even if user segments are compromised, they can't reach or spoof the HA heartbeat/VIP traffic.
5. **Monitor:** alert on unexpected gratuitous ARP for known VIP addresses outside of scheduled failover events — a sudden VIP MAC change with no corresponding failover log entry is a strong poisoning indicator.

8.3 Related Reading / Next Tools to Explore

- ✓ keepalived + VRRP fundamentals (to understand what "legitimate" gratuitous ARP looks like)
 - ✓ HSRP/VRRP hijacking techniques (Cisco-specific attack research)
 - ✓ Bettercap's `arp.spoof` and `net.probe` modules for a scriptable version of everything demoed manually with Ettercap
 - ✓ Responder + mitm6 (see Section 6.2) as the modern, AD-focused evolution of "control name resolution to control traffic"
-

09 Quick Reference Cheat Sheet

Attack Chain Summary

- | | |
|-----------------------------------|---|
| 1. Scan segment for hosts | → ettercap/bettercap host discovery, nmap -sn |
| 2. Confirm broadcast domain reach | → wireshark filter: arp |
| 3. ARP-poison victim <-> gateway | → ettercap Mitm > ARP Poisoning (bidirectional) |
| 4. Verify poisoning | → arp -a on victim, check MAC changed |
| 5. (Optional) layer DNS spoofing | → activate dns_spoof plugin + etter.dns records |
| 6. Verify DNS poisoning | → ipconfig /displaydns on victim |
| 7. Capture/modify traffic | → wireshark Follow TCP Stream, sslstrip |
| 8. Clean up | → stop MITM in ettercap to restore real ARP entries |

Command Cheat Sheet

Purpose	Command
View ARP table	<code>arp -a</code>
Flush Windows DNS cache	<code>ipconfig /flushdns</code>
View Windows DNS cache	<code>ipconfig /displaydns</code>
Launch Ettercap GUI	<code>sudo ettercap -G</code>
Enable IP forwarding (Linux)	<code>echo 1 > /proc/sys/net/ipv4/ip_forward</code>
Basic ARP spoof (dsniff)	<code>arp spoof -i eth0 -t <target> <gateway></code>
Edit ettercap DNS records	<code>nano /etc/ettercap/etter.dns</code>

Interview-Ready Talking Points

- ✓ Can explain *why* ARP/DNS lack authentication, not just *that* they're vulnerable
- ✓ Can name the DNS resolution order from memory and where each step can be attacked
- ✓ Knows the difference between Ettercap and Bettercap and when you'd reach for each
- ✓ Can describe Dynamic ARP Inspection as the primary real-world fix, not just "use a firewall"
- ✓ Understands that DNS spoofing on-path usually requires ARP poisoning first
- ✓ Can connect ARP poisoning concepts to gratuitous-ARP-based HA/load-balancer failover
- ✓ Knows Responder/mitm6 as the modern evolution of this attack family on Windows/AD networks

End of notes — CyberStr Red Team Internship Track · Prepared from ITProTV lecture transcript + trainer-level context